evidence-map

Where to look for evidence in GCP

The console tour, in flow order — input → transform → output → orchestration.

Everything below is written against `${TF_VAR_project_id}` in `${TF_VAR_region}`; the runs this repo captured were on `europe-west1`. Buckets are named `<project-id>-<role>`, so the prefix on your project is `${TF_VAR_project_id}-`. The live names come from:

```
terraform -chdir=terraform/envs/dev output buckets
```

Input data (`.DAT` / `.CHS` / `.ERR` / `.FLG`)

Cloud Storage → **Buckets** → **`<project>-landing`**. The mainframe extract lands here, one folder per `run_id`; the `.FLG` semaphore is written last by design.

Bucket	Role
`<project>-landing`	input extract from mainframe
`<project>-dataflow-temp` / `-dataflow-templates`	Dataflow staging + Flex Template specs
`<project>-json-out`	JSON producer output → handoff to the Loader team
`<project>-recon`	reconciliation / migrability reports
`<project>-tfstate`	Terraform state (not flow data)

Dataflow

Dataflow → **Jobs**: three Beam pipelines, one job per run — `file_processor`, `data_enrichment`, `json_producer`. Each job's **Logs** tab = worker logs, **Metrics** tab = `records_in` / `records_out` (the two-door counters).

Artifact Registry → **Repositories** → **`mig-dataflow`** — holds both the pod images the DAG runs (`build_java_images.sh`) and the Flex Template runner images (`make build-templates`).

IAM & Admin → **Service Accounts**: `dataflow-worker@<project>.iam.gserviceaccount.com` (worker SA), `composer-runner@…` (impersonates it). No `mig-` prefix — the account id is the module key.

BigQuery

BigQuery → **`<project>`** — three datasets:

Dataset	What lands here
`bq_extraction`	raw parsed records out of `file_processor`
`bq_transformation`	Dataform SQLX output
`bq_recon`	`run_ledger` (the balancing equation), `record_lineage` (every not-migrated record, named), `reject_log`

Query the equation for one run:

```
SELECT * FROM `<project>.bq_recon.run_ledger` WHERE run_id = 'initial-…'
```

One row per door: written / rejected. To go from the count to the records behind it:

```
SELECT door, stage, reason, COUNT(*) AS n
FROM `<project>.bq_recon.record_lineage`
WHERE run_id = 'initial-…'
GROUP BY door, stage, reason ORDER BY n DESC
```

Dataform

Dataform → Repositories → **`mig-000001-1`**. **Compilation Results** = `DataformCreateCompilationResult` output; **Workflow Invocations** = the actual runs. The SQLX models live in `dataform/definitions/*.sqlx`, pushed to a linked git remote (`make deploy-dataform`).

Optional sink — Managed Kafka

Managed Service for Kafka → Clusters → **`mig-kafka`**, two topics:

Topic	Who writes	Who reads
`target-system-target`	`json_producer` (the enriched TARGET JSON)	the Loader team
`target-system-confirmations`	`target-system-mock` — one `{runId, accountId, accountKey, confirmedAt}` per accepted write (HTTP 201)	`recon-service` — set-differences the keys against `account_target`, fails the run on an unconfirmed row (criterion 9)

Default `enable_kafka=false` and torn down between runs, so it is not visible unless `terraform apply -var=enable_kafka=true`. Three things the broker needs beyond the flag, all provisioned by the same apply:

- **A Serverless VPC Access connector** (`module.vpc_connector`, `mig-vpc-connector`) — Managed Kafka is VPC-internal, so the Cloud Run mock egresses through this connector to reach the broker. `egress = PRIVATE_RANGES_ONLY` routes only RFC1918 (the broker) through it.
- **`roles/managedkafka.client`** granted at the **project** level (not cluster level — the Terraform provider exposes no `google_managed_kafka_cluster_iam_member` resource, only `cluster` / `topic` / `acl`) to three service accounts: `dataflow-worker` (json_producer), `recon-service` (consumes confirmations),

`target-system-mock` (publishes confirmations). Without it the SASL_SSL/OAUTHBEARER handshake authenticates the SA but the broker refuses the connection as unauthorized.

- **`KAFKA_SECURITY_PROTOCOL=SASL_SSL`** env on the mock — flips its Java producer from PLAINTEXT (local redpanda) to SASL_SSL/OAUTHBEARER with `GcpTokenOauthCallbackHandler`.

Host-reachability constraint. `make smoke-gcp` runs recon-service and json_producer on the **host laptop** (inside the `mig-toolbox` podman container), not in the VPC. The VPC connector serves serverless GCP services (Cloud Run), not a host. So the host-side recon/json_producer **cannot reach VPC-internal Kafka** — only the Composer DAG (pods on Composer's GKE cluster in `mig-vpc`) runs these inside the VPC. A live criterion-9 green on GCP therefore requires the full Composer DAG path, not `make smoke-gcp`. See `runbook-gcp.md`.

Cloud Composer

Cloud Composer → **Environments** → **`mig-composer`**, behind `enable_composer` (default off — see the cost note in `runbook-gcp.md`). Inside: **DAGs** → **`mig_000001_1_migration`** is the migration DAG; **Logs** per task; **Airflow UI** link.

The DAG runs the 3 Beam pipelines + 2 Java apps as `KubernetesPodOperator` pods on Composer's GKE cluster, using the `mig-pipeline` Kubernetes ServiceAccount (created by `terraform/modules/composer_rbac`, annotated onto `dataflow-worker` via Workload Identity) — so failures show under **GKE** → **Workloads** in the Composer namespace, not just in Dataflow.

Output handoff + reconciliation

- **GCS `<project>-json-out`** — JSON producer output for the Loader team.
- **GCS `<project>-recon`** — `migrability_report` / `reconciliability_report` per run id.
- `loader-app` and `recon-service` run as pods, not as separate GCP products; their artefacts are the GCS objects above.

Cross-cutting

What	Where
Secrets (PGP key, Target System creds)	Security → Secret Manager (`make seed-secrets`)
API enablement (14 APIs)	APIs & Services → Enabled APIs
Network (VPC, NAT, inter-worker FW `tcp:12345-12346`)	VPC network → VPC networks, Firewall — a missing FW rule is the #1 cause of a Dataflow job that starts then never progresses
Billing	Billing — closed billing accounts block Composer/Kafka provisioning
Audit trail	Cloud Logging → Logs Explorer, filter `resource.type="dataflow_step"` or `"cloud_composer_environment"`

Quickest "where is everything" sweep

```
        terraform -chdir=terraform/envs/dev output
        gcloud storage buckets list --project="$TF_VAR_project_id"
        gcloud dataflow jobs list --region="$TF_VAR_region" --project="$TF_VAR_project_id"
        bq ls --project_id="$TF_VAR_project_id"
        gcloud composer environments list --locations="$TF_VAR_region" --
project="$TF_VAR_project_id"
```

Captured evidence of real runs (all 9 acceptance criteria passing), newest first: `evidence/gcp-composer-20260823/` (full DAG green **with Managed Kafka on** — criterion 9 closed on live confirmations, 50/50 TARGET rows), `evidence/gcp-smoke-20260822/` (two-door + rename HEAD re-verified via `make smoke-gcp`), `evidence/gcp-composer-20260820/` (full DAG green on Cloud Composer 2), `evidence/gcp-composer-20260818/` and `evidence/gcp-run-20260804/` / `evidence/gcp-composer-20260805/` — those are the artefacts to compare against the live console.